

大模型与多模态生成模型推理框架技术图谱

面向工程选型的推理框架报告，覆盖在线服务引擎、本地与边缘运行时、分布式推理编排、结构化输出、多模态模型服务，以及图像和视频生成模型推理 workflow。

Winston

资料口径截至 2026-05-04

Primary repositories · official documentation · technical papers

| 目录

01	执行摘要	推理框架的核心问题不是 API 包装
02	分类框架	按系统层、部署边界和模型模态拆分
03	在线推理服务引擎	vLLM、SGLang、TensorRT-LLM、TGI、LMDeploy
04	分布式服务与运行平台	Dynamo、Triton、Ray Serve
05	本地、边缘与低成本推理	llama.cpp、Ollama、MLC、MLX、ONNX、OpenVINO
06	多模态与结构化输出	VLM 服务、约束解码、工具调用
07	图像与视频生成推理	Diffusers、ComfyUI、视觉生成加速
08	选型方法与边界	场景入口、指标、风险清单

执行摘要

推理框架的技术价值，主要体现在请求调度、KV cache 管理、显存占用、内核效率、模型格式、量化策略、服务协议和多租户运维能力。把推理框架理解成一个 OpenAI-compatible API server，会低估在线服务、长上下文、多模态和图像视频生成的系统复杂度。

FINDING 01

服务引擎优先于接口外观

高并发文本服务的主要瓶颈通常不是 HTTP，而是 prefill、decode、KV cache、连续批处理和调度策略。vLLM、SGLang、TensorRT-LLM、LMDeploy 等项目的差异，集中在这些底层能力。

FINDING 02

本地推理是格式与硬件问题

llama.cpp、Ollama、MLC LLM、MLX-LM、OpenVINO GenAI、ONNX Runtime GenAI 并非同一类产品。它们分别围绕 GGUF、应用封装、机器学习编译、Apple Silicon、Intel PC 和 ONNX 生态形成能力边界。

FINDING 03

长上下文放大系统差异

长上下文请求会放大显存、KV cache、前缀复用、prefill/decode 拆分和路由策略的影响。对 RAG、代码仓库理解、长文档分析和推理型模型而言，调度策略常常比单次模型速度更重要。

FINDING 04

视觉生成需要单独评估

图像与视频生成推理不是文本服务的附属功能。它涉及扩散管线、采样器、ControlNet/LoRA、节点 workflow、显存复用、队列调度和模型权重组织，应作为独立的推理工作负载评估。

READING POSITION

本报告面向需要选择、整合或复核推理框架的技术负责人和工程团队。正文不试图给出单一总榜，而是以场景入口判断框架：高 QPS 在线聊天、长上下文 RAG、代码助手、多模态 API、本地桌面应用、边缘设备、图像 workflow、视频生成任务和企业 GPU 集群，对框架的要求并不相同。

推理框架的判断指标

指标	含义	选型意义
TTFT	time to first token, 首 token 延迟。	影响交互体感, 长上下文和 prefill 成本高时尤其关键。
TPOT / ITL	每 token 生成时间或 token 间延迟。	影响流式输出速度和多轮会话体验。
吞吐	单位时间处理的请求或 token 数。	决定单位 GPU 的服务成本, 需与延迟 SLA 同时评估。
显存效率	权重、KV cache、激活、临时 buffer 与碎片的综合占用。	决定可服务模型规模、batch 大小、上下文长度和并发上限。
格式与量化	safetensors、GGUF、ONNX、OpenVINO IR、TensorRT engine、EXL2 等。	影响模型迁移、准确率、推理速度、硬件可用性和运维复杂度。
协议与运维	OpenAI-compatible API、SSE、metrics、tracing、多租户、安全控制。	决定能否进入生产链路, 而不是只在单机 demo 中运行。

分类框架

推理框架应按系统层级理解：接口层负责请求语义，服务层负责排队与调度，执行层负责算子和内存，格式层决定模型如何进入运行时，硬件层决定可用性能边界，生成工作流层承载图像、视频和复杂管线。



图 1：推理框架的系统层级。不同项目可以横跨多层，但选型时应先明确核心职责。

纳入与排除边界

类别	纳入条件	本报告处理方式
推理服务引擎	直接负责模型推理执行、请求调度、显存与 KV cache 管理。	主体分析。
分布式推理平台	面向多节点、多 GPU、多模型或大规模生产服务的编排系统。	按服务边界分析，不与单节点引擎混排。
本地运行时	面向桌面、边缘、移动端或低成本硬件的模型运行工具链。	按模型格式与硬件路径分析。
生成模型工作流	承载图像、视频、音频生成 pipeline 或节点图。	单独章节讨论。
训练框架	以训练、微调、RL 后训练为主。	不纳入主体。
Agent 与应用框架	以工具调用、规划、记忆、工作流应用为主。	只在接口边界中提及。

BOUNDARY PRINCIPLE

边界划分的目的不是否认上下游组件的重要性，而是避免把不同层级的问题混在同一张表中比较。推理服务引擎回答“如何执行请求”，分布式平台回答“如何组织一组引擎”，本地运行时回答“如何在受限设备上运行模型”，生成工作流回答“如何组合多阶段视觉生成过程”。

在线推理服务引擎

在线服务引擎是现阶段大模型推理框架竞争最集中的区域。核心问题包括：如何把动态长度请求高效批处理，如何控制 KV cache 占用，如何降低首 token 延迟，如何支持量化与多 GPU，如何把结构化输出、多模态输入和工具调用纳入稳定服务协议。

主干框架对照

框架	关键定位	适合场景	需要特别核查
vLLM	高吞吐、显存高效的 LLM inference and serving engine，以 PagedAttention、连续批处理、prefix caching、并行策略和 OpenAI-compatible server 构成主干。	通用在线文本服务、长上下文 RAG、多租户模型服务、需要广泛模型支持的生产环境。	目标模型架构、量化格式、多模态 processor、结构化输出后端、特定硬件插件的成熟度。
SGLang	面向大模型和多模态模型的高性能服务框架，强调 RadixAttention、结构化输出、复杂语言模型程序、agentic 和多轮任务的执行效率。	多轮交互、RAG pipeline、结构化输出、复杂提示程序、多模态服务、推理型模型服务。	与目标模型、采样策略、grammar backend、并行模式和部署形态的兼容性。
TensorRT-LLM	NVIDIA GPU 优化栈，提供 PyTorch 原生 API、Python/C++ runtime、定制 kernel、量化、MoE、speculative decoding 和多 GPU/多节点能力。	NVIDIA 数据中心 GPU、性能优先、需要深度优化和企业级硬件栈协同的生产服务。	engine 构建流程、版本与 CUDA/驱动匹配、模型支持、调优复杂度、telemetry 与企业合规要求。
TGI	Hugging Face 出品的文本生成推理服务，提供 launcher、SSE streaming、metrics、tensor parallelism、连续批处理和常见量化路径。	既有 Hugging Face 部署链路、模型库集成、已有 TGI 生产资产维护。	公开仓库目前为 archived 状态，新项目应重点复核维护策略和替代路径。
LMDeploy	OpenMMLab/InternLM 生态的压缩、部署和服务工具，包含 TurboMind 与 PyTorch engine，覆盖 LLM/VLM 部署。	中文开源模型生态、InternLM/OpenMMLab 相关项目、需要压缩与服务一体化。	非主流模型族、量化路径、VLM 输入链路和部署平台限制。
LightLLM / Aphrodite	更偏专门化和研究/社区实践的推理服务框架，分别强调轻量、高性能调度或 vLLM 相关技术路线延展。	特定模型、研究复现、社区平台服务、需要实验性调度或 kernel 能力的团队。	维护节奏、生产支持、license、与主流生态的迁移成本。

vLLM: 通用在线服务基准线

vLLM 的核心贡献是把 KV cache 管理从静态、易碎片化的显存分配，转向更接近虚拟内存思想的页式管理。PagedAttention 论文指出，LLM serving 的吞吐受限于动态增长的 KV cache；vLLM 在此基础上实现了连续批处理、prefix caching、chunked prefill、并行策略、量化路径和 OpenAI-compatible API server，使其成为通用在线服务选型的基准线。

在工程上，vLLM 的优势不是某一个模型的峰值 benchmark，而是模型覆盖、部署接口、社区活跃度和系统能力的组合。若目标是构建通用文本服务、RAG 服务或内部模型网关，vLLM 通常是首个应验证的选项。需要注意的是，多模态、结构化输出、工具调用和非 CUDA 硬件的成熟度依具体版本、模型与插件而变化，不能只看项目总能力说明。

SGLang: 复杂程序与结构化执行

SGLang 的定位不是简单替代 vLLM，而是在复杂语言模型程序和多轮任务上更积极地组织执行。其论文将系统拆分为 frontend language 与 runtime，runtime 使用 RadixAttention 复用 KV cache，并引入面向结构化输出的优化。对 agent 控制、few-shot、多轮对话、RAG、JSON decoding 和多模态任务，SGLang 的设计更贴近“一个请求包含多次模型调用和控制流”的现实。

在选型时，SGLang 的价值应围绕任务结构判断：如果业务请求存在大量共享前缀、多分支生成、结构化输出、工具调用或多模态输入，SGLang 值得优先压测。若只是单轮通用聊天服务，则需要与 vLLM、TensorRT-LLM、LMDeploy 在同一模型、同一 SLA、同一硬件下比较。

TensorRT-LLM: 性能优先的 NVIDIA 路径

TensorRT-LLM 是 NVIDIA 推理栈中最重要的大模型服务组件之一。官方仓库将其描述为优化 LLM 与视觉生成推理的开源库，包含 attention、GEMM、MoE 等定制 kernel，以及 prefill/decode 拆分、expert parallelism、speculative decoding 等运行时优化。它与 Triton、Dynamo、NIM 等 NVIDIA 生态组件有天然衔接。

该路径的判断标准应更接近系统工程：目标 GPU 代际、CUDA 与驱动版本、engine 构建流程、模型支持矩阵、量化精度、峰值吞吐、团队调优能力和生产运维成本。对于大规模 NVIDIA GPU 集群，TensorRT-LLM 往往具备最强的性能上限；对于频繁切换模型、快速验证和跨硬件部署的团队，其复杂度需要纳入总成本。

TGI 与 LMDeploy: 生态型服务工具

TGI 曾是 Hugging Face 生态中非常重要的生产服务工具，具备 launcher、tensor parallelism、SSE streaming、continuous batching、metrics、tracing、quantization 和 Messages API 等能力。由于公开仓库当前处于 archived 状态，既有部署仍可依据官方文档维护，新项目则应谨慎评估维护路径、版本风险和替代方案。

LMDeploy 则更贴近 OpenMMLab 与 InternLM 生态，强调 LLM 压缩、部署与服务一体化。其 TurboMind 和 PyTorch engine 可覆盖多类 LLM/VLM 服务场景，适合已经采用相关模型和工具链的团队。选型重点不应是“是否比某框架快”，而是目标模型、量化、VLM 输入、服务接口和运维栈是否匹配。

分布式服务与运行平台

当单节点服务能力不足时，问题会从“如何跑得快”转向“如何调度一组推理引擎”。多节点推理需要处理路由、扩缩容、缓存复用、故障迁移、跨节点通信、模型冷启动和资源隔离。此时 vLLM、SGLang、TensorRT-LLM 等引擎仍然重要，但上方还需要服务平台。

代表平台

平台	系统职责	与推理引擎的关系	适合场景
Dynamo	数据中心规模的分布式推理编排，强调 disaggregated serving、KV-aware routing、多层 KV cache、SLA 规划和自动扩缩容。	位于 vLLM、SGLang、TensorRT-LLM 等引擎上方，不替代单节点引擎。	多 GPU/多节点、推理型模型、长上下文、reasoning、VLM 和视频生成负载。
Triton Inference Server	NVIDIA 通用推理服务平台，覆盖云端与边缘，提供模型仓库、后端、动态批处理、metrics 和多模型服务能力。	可承载 TensorRT-LLM backend、vLLM backend 或其他模型后端。	企业推理平台、异构模型服务、需要标准化运维和 NVIDIA 生态集成的环境。
Ray Serve LLM	Ray Serve 面向 LLM 的服务层，强调自动扩缩容、负载均衡、多节点多模型、OpenAI-compatible API、多 LoRA 和观测。	通常作为 orchestration 层，底层可接 vLLM、SGLang 等引擎。	已经使用 Ray 生态、需要 Pythonic 分布式应用、模型服务与业务逻辑共同编排的团队。

分布式推理的真实瓶颈

分布式推理不是简单地增加 GPU。Prefill 与 decode 的计算形态不同：prefill 更像大矩阵吞吐问题，decode 更受小步迭代和 KV cache 访问影响。长上下文与多轮对话又会引入 prefix cache 和 KV cache 迁移问题。若路由策略不了解缓存复用，集群可能在拥有更多 GPU 的同时重复计算更多 prefill。

Dynamo 的设计重点正是把单节点引擎编排为多节点系统，提供 prefill/decode 拆分、KV-aware routing、多层 KV cache 和 SLA-aware planning。Triton 更适合承担标准化推理平台角色，Ray Serve 更适合把模型服务嵌入分布式 Python 应用与业务 workflow。三者不是同一层级的替代关系，选型时应先明确平台希望控制的资源边界。

何时只需要服务引擎

适合：单模型、单节点或少量 GPU，流量规模可由引擎自身调度解决，团队已有反向代理、监控和部署体系。

风险：随着模型增大、上下文变长或租户增多，手写路由和扩缩容逻辑会快速失控。

何时需要编排平台

适合：多模型、多节点、多 LoRA、多租户、长上下文和 reasoning 服务，需要成本、延迟和利用率同时优化。

风险：平台引入的复杂度只有在规模足够大、SLA 足够明确时才值得承担。

本地、边缘与低成本推理

本地推理框架的核心不是“是否能聊天”，而是权重格式、量化精度、内存带宽、硬件后端和应用封装。桌面端、移动端、浏览器、企业 PC、消费级 GPU 与 CPU-only 环境各有约束，不能用于在线服务框架的指标直接评估。

本地推理框架对照

框架	主要路径	优势	边界
llama.cpp	C/C++、ggml、GGUF、多硬件后端。	依赖少，覆盖 CPU、CUDA、Metal、Vulkan、SYCL 等路径，适合本地和嵌入式场景。	生产级多租户服务能力需另行封装；多模态和新架构支持依版本而定。
Ollama	本地模型管理、CLI、REST API，底层依托 llama.cpp。	安装和使用门槛低，适合个人、团队内部和桌面应用原型。	更像产品化运行入口，不同于底层推理引擎。
MLC LLM	机器学习编译与 MLCEngine。	覆盖 NVIDIA/AMD/Apple/Intel、WebGPU、iOS、Android 等平台，适合跨端部署。	模型编译、格式转换和平台适配成本需要评估。
MLX-LM	Apple Silicon 上的 MLX 生态。	对 Mac 本地生成、量化、prompt cache 和 Apple 统一内存路径友好。	硬件边界明确，主要服务 Apple Silicon。
ONNX Runtime GenAI	ONNX Runtime 上的生成循环、KV cache、sampling、grammar。	适合已有 ONNX/Windows/跨平台推理资产，便于进入设备端链路。	需要模型导出、算子支持和性能验证。
OpenVINO GenAI	OpenVINO Runtime 上的 C++/Python 生成模型 pipeline。	适合 PC、笔记本和 Intel 生态，支持 continuous batching、prefix caching、speculative decoding 等生成优化。	应以目标 Intel 硬件与目标模型实测为准。
KTransformers	CPU-GPU 异构推理，尤其关注 MoE 模型。	在有限显存场景下通过 CPU/GPU 分层和 MoE 优化扩展可运行模型规模。	更偏专门场景，部署复杂度和模型适配需要投入。
ExLlamaV2 / mistral.rs	消费级 GPU 高速本地推理、Rust/CUDA/Metal 等路径。	适合本地高性能、量化权重和开发者工具链探索。	生态接口、模型覆盖和生产服务能力需要逐项确认。

本地推理的四个判断问题

- 模型是否有合适格式。GGUF、EXL2、MLX、ONNX、OpenVINO IR 和 safetensors 的转换链路不同，量化后准确率也不同。
- 目标硬件瓶颈是什么。CPU、消费级 GPU、Apple Silicon、Intel GPU/NPU、移动端和浏览器的瓶颈差异很大，不能只看参数量。
- 应用需要什么接口。桌面应用需要模型管理和 REST API，嵌入式应用需要低依赖库，企业 PC 需要可控部署与更新策略。
- 长上下文是否必要。本地长上下文会迅速放大内存占用，prompt cache、rotating KV cache 和量化策略需要提前验证。

SELECTION PRINCIPLE

本地推理不应盲目追求“最大模型”。更合理的路径是先确定可接受延迟、内存上限、离线要求、隐私边界和目标设备，再选择权重格式与运行时。Ollama 适合快速应用入口，llama.cpp 适合底层可控部署，MLC LLM 适合跨端，MLX-LM 适合 Apple Silicon，OpenVINO/ONNX Runtime GenAI 适合企业 PC 与标准化设备链路。

本地部署画像

部署画像	优先目标	框架入口	验收重点
个人开发者桌面	低门槛、模型管理、离线使用。	Ollama、llama.cpp、MLX-LM。	安装路径、模型下载、内存占用、流式响应。
企业知识助手	可控部署、隐私边界、稳定 API。	llama.cpp server、OpenVINO GenAI、ONNX Runtime GenAI。	版本治理、审计、模型更新和设备兼容。
跨端应用	同一模型跨桌面、移动端和浏览器。	MLC LLM、ONNX、平台原生运行时。	编译链路、包体积、端侧性能和热更新策略。
有限显存大模型	在消费级硬件上运行更大模型或 MoE。	KTransformers、ExLlamaV2、llama.cpp。	量化损失、CPU/GPU 分层、上下文长度和稳定性。

多模态与结构化输出

多模态推理和结构化输出是 2026 年推理框架必须认真处理的能力。前者要求 tokenizer、processor、图像/视频预处理和模型架构一致；后者要求在 decode 阶段控制 token 选择，而不是生成后再用正则表达式修补。

多模态推理的工程边界

VLM/MLLM 服务链路通常包含图像或视频解码、resize/crop、视觉 encoder、token 插入、chat template、LLM decode 和后处理。框架宣称支持多模态，并不意味着所有视觉语言模型都可直接进入生产。目标模型的 processor、输入尺寸、batch 规则、显存占用、流式输出和多图/视频输入都需要单独验证。

vLLM、SGLang、LMDeploy、mistral.rs 等项目都在多模态方向扩展，但能力边界取决于模型族和版本。对于多模态服务，选型表应增加三个维度：输入处理是否与训练配置一致，图像/视频 token 对上下文预算的影响是否可控，线上异常输入是否会破坏服务稳定性。

结构化输出从“后处理”转向“约束解码”

组件	定位	适合问题	注意事项
llguidance	面向 LLM 的 constrained decoding / structured output 库，支持 JSON schema、正则和上下文无关文法。	需要在生成过程中强制输出合法 JSON、DSL 或工具参数。	需要确认推理引擎集成状态、tokenizer 适配和 grammar 复杂度。
Outlines	面向结构化生成的 Python 库，可将类型、Pydantic model、枚举等约束映射到生成过程。	抽取、分类、表单填充、结构化业务对象生成。	更偏开发接口和约束表达，生产部署需与底层服务引擎配合。
FlashInfer	面向 LLM serving 的高性能 GPU kernel 库，覆盖 attention、GEMM、MoE、sampling 等。	作为 vLLM、SGLang、TensorRT-LLM、TGI 等系统的底层内核组件。	一般不是业务团队直接部署的完整服务框架，但会影响引擎性能上限。

结构化输出的质量边界

结构化输出解决的是格式有效性，不自动解决事实正确性、业务校验和安全策略。即使 JSON schema 合法，字段值仍可能不符合业务语义。因此生产系统应保留 schema 校验、业务规则校验、失败重试、审计日志和人工兜底。约束解码的价值在于减少格式错误和解析失败，使下游系统获得更稳定的输入。

图像与视频生成推理

AIGC 图像与视频生成推理是另一条技术路线。文本 LLM 服务围绕 token decode 和 KV cache 展开，扩散与视频生成则围绕 pipeline、scheduler、latent、control condition、LoRA、显存复用和队列调度展开。把两者放在同一张“LLM serving”表中，会遗漏关键工程因素。

Diffusers：生成模型 pipeline 基础库

Diffusers 是 Hugging Face 生态中最重要的扩散模型库之一，官方定位覆盖 image、video、audio generation in PyTorch。它提供 pipeline、scheduler 和模型组件，适合研究、原型、模型集成和服务端自定义推理。对于需要掌控模型结构、采样器、加速策略、LoRA 和 ControlNet 的团队，Diffusers 是比图形界面更底层的工程入口。

Diffusers 的优势在于透明、可组合、与 Hugging Face Hub 连接紧密；边界在于生产服务能力需要自行补齐，包括 API、队列、缓存、并发控制、模型热加载、显存隔离和监控。

ComfyUI：节点图工作流与创作型推理

ComfyUI 将扩散模型推理组织为节点图，适合复杂图像工作流、Hires fix、inpainting、ControlNet、upscale、模型组合和创作工具链。它的价值不只是 GUI，而是把生成过程显式拆解为可复用、可分享、可调试的图结构。对于设计、内容生产和复杂工作流验证，ComfyUI 的表达能力强于线性脚本。

将 ComfyUI 用作生产服务时，需要额外关注队列调度、工作流版本管理、自定义节点供应链、模型权重管理、显存峰值、API 稳定性和安全隔离。若目标是大规模 API 服务，通常需要在 ComfyUI 工作流之外构建更严格的任务系统。

视觉生成推理的选型表

场景	优先入口	理由	补充能力
模型研究与 pipeline 集成	Diffusers	组件透明，可控制 scheduler、模型模块、LoRA 和优化策略。	自行构建服务 API、队列和监控。
复杂图像工作流	ComfyUI	节点图适合组合 ControlNet、IP-Adapter、upscale、inpainting 和多阶段生成。	需要治理自定义节点和工作流版本。
高性能 NVIDIA 部署	TensorRT / TensorRT-LLM 视觉生成路径	适合对延迟、吞吐和 GPU 利用率有严格要求的环境。	需要 engine 构建、算子支持和模型兼容验证。
端侧图像生成	OpenVINO / ONNX / Core ML / MLC 等路径	面向 PC、移动端和边缘设备的格式与硬件优化。	需接受模型规模、分辨率和速度约束。
视频生成	Diffusers 或专用项目 pipeline	视频任务受帧数、分辨率、时序一致性和显存影响更大。	队列、断点、存储、预览和失败恢复比文本服务更重要。

选型方法与边界

推理框架选型应从业务负载进入，而不是从项目名进入。一个成熟的选型过程至少需要定义模型族、上下文长度、输入模态、延迟目标、吞吐目标、硬件预算、量化精度、服务协议、观测要求、合规约束和未来迁移成本。

场景入口矩阵

场景	优先验证	关键指标	常见误区
高 QPS 在线聊天	vLLM、SGLang、TensorRT-LLM、LMDeploy	吞吐、TTFT、TPOT、显存利用率、metrics。	只测单请求速度，不测并发与 SLA。
长上下文 RAG	vLLM、SGLang、Dynamo	prefill 成本、prefix cache、KV cache 占用、路由策略。	忽略检索结果长度和共享前缀复用。
推理型模型与代码助手	SGLang、vLLM、TensorRT-LLM、Dynamo	长输出、工具调用、结构化输出、多轮状态、decode 稳定性。	只按短 prompt 聊天 benchmark 判断。
多模态 API	SGLang、vLLM、LMDeploy、mistral.rs	processor 一致性、图像/视频 token 预算、批处理规则。	只看模型是否列在 support list。
本地桌面应用	Ollama、llama.cpp、MLX-LM、mistral.rs	安装成本、模型管理、内存占用、离线能力。	把桌面体验和生产服务混为一谈。
边缘与企业 PC	OpenVINO GenAI、ONNX Runtime GenAI、MLC LLM、llama.cpp	目标硬件、格式转换、低依赖、更新策略。	只在开发机 GPU 上验证。
图像生成 workflow	ComfyUI、Diffusers	workflow 表达、显存峰值、节点供应链、队列。	用纯文本服务指标评估扩散模型。
企业 GPU 集群	Dynamo、Triton、Ray Serve、TensorRT-LLM、vLLM	多节点调度、扩缩容、故障迁移、观测、成本。	先定平台，再倒推模型和 SLA。

采用前检查清单

模型与性能

- 目标模型架构、tokenizer、chat template、processor 是否被稳定支持。
- 上下文长度、输出长度、并发量和批处理策略是否贴近真实流量。
- 量化格式是否有可接受的准确率损失和可复现转换流程。
- 首 token、token 间延迟、吞吐和显存峰值是否同时达标。

服务与治理

- API 协议、流式输出、错误码、超时、取消请求和限流是否完整。
- metrics、tracing、日志、审计、成本归因和容量规划是否可落地。
- license、权重许可、自定义节点、第三方 kernel 与供应链风险是否可控。
- 升级策略、回滚策略、模型热更新和多租户隔离是否明确。

资料口径与后续复核

高风险信号	可能后果	建议动作
只提供单请求 demo，没有并发压测。	上线后 TTFT、TPOT 和排队延迟不可控。	按真实 prompt 长度、输出长度和并发分布重跑 benchmark。
模型支持依赖未合并补丁或自定义分支。	升级困难，安全修复和上游新能力难以继承。	确认 upstream 状态、维护人和回滚策略。
量化格式只看速度，不看任务质量。	业务准确率、工具调用稳定性或结构化输出质量下降。	建立目标任务评测集，比较原始权重与量化权重。
多模态输入缺少异常治理。	大图、坏文件、长视频或恶意输入导致服务抖动。	限制输入尺寸、格式、时长和预处理耗时，并记录失败样本。

本报告依据截至 2026-05-04 的公开一手资料、官方文档、论文页面和仓库元数据整理。由于推理框架迭代速度快，报告中的框架定位应被视为阶段性技术图谱，而不是永久结论。进入正式选型前，应以目标模型、目标版本、目标硬件和真实流量重跑 benchmark，并复核 license、依赖链和维护状态。

完整来源与仓库元数据见 sources.md 与 data/repository-metadata.tsv。训练框架、Agent 应用框架、云托管 API、评测平台和向量数据库并非本文主体，后续若需要构建完整大模型工程栈，应分别形成独立报告。